



**FACULTAD DE
INGENIERÍA**

UNIVERSIDAD DA VINCI
DE GUATEMALA

Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: Ingeniería En Sistemas



**FACULTAD DE
INGENIERÍA**

UNIVERSIDAD DA VINCI
DE GUATEMALA

“Examen Final”

Bernardo Emiliano Canas Calán

202401637

Curso: Desarrollo Web

Guatemala, Diciembre de 2025



Propuesta de Proyecto de Innovación Web: "AgendCanas"

Este proyecto tiene como objetivo demostrar la capacidad de conceptualizar, arquitecturar y planificar un proyecto de software profesional, actuando como un ingeniero capaz de tomar decisiones técnicas y dimensionar esfuerzos reales.

Definición del Problema y Solución

El Problema

El principal problema identificado es la **dificultad que enfrentan los estudiantes universitarios para gestionar de manera centralizada y eficiente su carga académica** (tareas, fechas límite, exámenes) de múltiples cursos, sumado a la **procrastinación** y la **falta de seguimiento** de su progreso real. Las herramientas existentes (calendarios, listas de tareas) son genéricas y no están optimizadas para el contexto académico.

La Solución

Nuestra aplicación web, "**AgendCanas**", es un **gestor de carga académica inteligente** y un **planificador de estudios** que utiliza técnicas de productividad (como el método Pomodoro adaptado) para ayudar a los estudiantes a:

1. **Centralizar** todas sus tareas y exámenes por curso.
2. **Priorizar** automáticamente las tareas basándose en la fecha límite, peso en la nota final y esfuerzo estimado.
3. **Generar sesiones de estudio** enfocadas, con temporizador Pomodoro integrado y registro de tiempo.

Justificación

EduFocus es una idea innovadora y necesaria porque aporta valor al enfocarse en laproductividad específica del estudiante universitario:

- **Gestión de Peso Académico:** Permite al estudiante ver no solo las tareas, sino su impacto real en la nota final, motivando la priorización.
 - **Antiprogastinación Activa:** El uso del método Pomodoro dentro de la aplicación fomenta el inicio de la tarea y la gestión del tiempo de descanso.
 - **Centralización:** Reduce la dispersión de información en múltiples plataformas (emails, LMS, notas físicas) en un solo dashboard.
-



2. Propuesta Técnica

La arquitectura será un diseño de microservicios ligero basado en la división estándar de Frontend (Cliente) y Backend (Servidor) con una base de datos relacional para garantizar la integridad de los datos académicos.

A. Frontend

Para el prototipo se requiere utilizar **Vue.js**.

Criterio	React	Vue.js	Angular	Justificación de Elección
Curva de Aprendizaje	Media	Baja	Alta	Vue.js es más accesible y rápido para un MVP, permitiendo enfocarse en la funcionalidad.
Performance	Alta	Alta	Media	La performance de Vue es comparable y su reactividad es muy eficiente.
Integración (Ecosistema)	Muy Alta	Media	Alta	Para un proyecto de este tamaño, el ecosistema de Vue es más que suficiente y su integración es sencilla.

B. Backend

Lenguaje/Framework: **Node.js** con el framework **Express.js**.

- **Justificación:** Node.js es ideal para aplicaciones I/O-bound (como una web de gestión) debido a su modelo asíncrono y *single-threaded*. Esto permite un manejo rápido de múltiples peticiones (muchos estudiantes) con recursos eficientes.

Publicación (Deployment): **Vercel** para el Frontend y **AWS EC2/Elastic Beanstalk** para el Backend.

- **Justificación:** Vercel ofrece un *deploy* rápido y gratuito para proyectos Vue (estáticos). AWS es robusto y escalable para el *backend* de Express, permitiendo la escalabilidad futura.

C. Persistencia de Datos

Elección: **SQL (PostgreSQL)**.

Justificación: La naturaleza de los datos académicos requiere altos niveles de consistencia e integridad relacional. Necesitamos establecer relaciones estrictas entre Usuarios, Cursos, Tareas, y Sesiones de Estudio. Por ejemplo, una Tarea siempre debe pertenecer a un Curso y un Curso a un Usuario. Una base de datos relacional garantiza que no se introduzcan datos inconsistentes.



Diagrama de Base de Datos

El esquema de la base de datos (simplificado) es el siguiente:

- Usuario (ID, Nombre, Email, Contraseña).
- Curso (ID, Nombre, Código, ID_Usuario (FK)).
- Tarea (ID, Título, Fecha_Limite, Peso_Nota, Esfuerzo_Estimado, ID_Curso (FK)).
- Sesion_Estudio (ID, Fecha, Duración_Real, Metodo_Usado, ID_Tarea (FK)).



3. Diseño de Interfaz de Programación (API)

Utilizaremos una API **RESTful** con formato **JSON** para la comunicación entre el Frontend (Vue) y el Backend (Express).



Endpoints Principales Documentados

Siguiendo el formato requerido:

Endpoint	Método HTTP	Descripción	Request Body (Ej.)	Response Body
/api/tareas	POST	Crea una nueva tarea para un curso.	{"titulo": "Ensayo Final", "curso_id": 4, "fecha_limite": "2026-01-15", "peso_nota": 30}	{"status": 201, "tarea_id": 10}
/api/tareas/{id}	GET	Obtiene los detalles de una tarea específica.	(Ninguno)	{"id": 10, "titulo": "Ensayo Final", "estado": "pendiente", ...}
/api/tareas/{id}	PUT/ PATCH	Actualiza el estado o detalles de una tarea.	{"estado": "completada"}	{"status": 200, "message": "Tarea actualizada"}

4. Planificación y Costos

A. Estimación de Esfuerzo

Utilizando una técnica de desglose de tareas para estimar el esfuerzo real

Tarea Principal	Desglose de Tareas (Ej.)	Horas Estimadas
Arquitectura/Setup	Config. DB, Repositorios, Conexión Inicial.	12 hrs
Frontend: Auth/Dashboard	Rutas, UI de Login, Diseño de Dashboard principal.	24 hrs
Backend: Tareas CRUD	Endpoints de Creación, Lectura, Actualización y Eliminación.	30 hrs
Frontend: Módulo Tareas	Componentes de Lista de Tareas, Formulario de Creación.	20 hrs
Módulo Pomodoro	Lógica del Temporizador, Tracking de tiempo.	25 hrs
Deployment	Config. Vercel, AWS EC2, Dominio.	10 hrs
Total Horas Bruto		121 hrs
Buffer (20%)	121×0.20	24.2 hrs



B. Presupuesto

1. Costo por Hora (Q.): Asumo un costo competitivo de Q. 75.00/hora.

2. Costo Total de Desarrollo:

- Costo Total = Horas Totales × Costo Hora
- Costo Total = 145.2 hrs×Q.75.00/hr
- Costo Total Estimado = Q. 10,890.0

Recursos Útiles Consultados y Recomendados

Para la documentación de la API, se recomienda usar Swagger Editor para visualizar el contrato, y para el seguimiento del tiempo durante el desarrollo se puede usar Toggl Track. Para el diagrama de la base de datos se utilizó la herramienta dbdiagram.io

Documentación API - AgendCanas

Base URL: <http://localhost:3000/api/v1>

Endpoint 1: Obtener lista de cursos

<http://localhost:3000/api/v1/tasks>

Retorna todas las tareas del usuario con información completa del curso asociado, ordenadas por prioridad y fecha límite.

Request

api/v1/tasks

Host: localhost:3000

Response Exitosa

```
{
  "success": true,
  "count": 3,
  "data": [
    {
      "id_curso": 6,
      "id_usuario": 1,
      "nombre_curso": "Desarrollo Web",
      "codigo_curso": "DES15",
      "color": "#4ECDC4",
      "creditos": 5,
```



```
"fecha_creacion": "2025-12-07T00:04:56.514Z",
"total_tareas": "1",
"tareas_pendientes": "1"
},
{
  "id_curso": 7,
  "id_usuario": 1,
  "nombre_curso": "Lenguajes Formales y Automatas",
  "codigo_curso": "FC89",
  "color": "#4ECDC4",
  "creditos": 3,
  "fecha_creacion": "2025-12-07T00:05:18.284Z",
  "total_tareas": "0",
  "tareas_pendientes": "0"
},
{
  "id_curso": 5,
  "id_usuario": 1,
  "nombre_curso": "Matemática IV",
  "codigo_curso": "MAT08",
  "color": "#FF6B6B",
  "creditos": 3,
  "fecha_creacion": "2025-12-06T23:16:54.056Z",
  "total_tareas": "1",
  "tareas_pendientes": "1"
}
]
}
```

Response de Error (500)

```
{
  "success": false,
  "message": "Error al obtener las tareas",
  "error": "Connection refused"
}
```

Endpoint 2: Obtener lista de tareas

http://localhost:3000/api/v1/tasks

Retorna todas las tareas del usuario con información completa del curso asociado, ordenadas por prioridad y fecha límite.



Request

/api/v1/tasks

Host: localhost:3000

Response Exitosa

```
{
  "success": true,
  "count": 2,
  "data": [
    {
      "id_tarea": 5,
      "titulo": "Examen FINAL",
      "descripcion": "Descripcion",
      "fecha_limite": "2025-12-08T05:59:00.000Z",
      "peso_nota": "30.00",
      "esfuerzo_estimado": 60,
      "prioridad": "urgente",
      "estado": "pendiente",
      "tiempo_trabajado": 0,
      "id_curso": 6,
      "nombre_curso": "Desarrollo Web",
      "codigo_curso": "DES15",
      "color": "#4ECDC4",
      "dias_restantes": 0
    },
    {
      "id_tarea": 7,
      "titulo": "Ejercicios del 1 al 15",
      "descripcion": "Subir ejercicios",
      "fecha_limite": "2025-12-11T06:00:00.000Z",
      "peso_nota": "20.00",
      "esfuerzo_estimado": 45,
      "prioridad": "media",
      "estado": "pendiente",
      "tiempo_trabajado": 0,
      "id_curso": 5,
      "nombre_curso": "Matemática IV",
      "codigo_curso": "MAT08",
      "color": "#FF6B6B",
      "dias_restantes": 2
    }
  ]
}
```



Endpoint 3: Registrar sesión Pomodoro

http://localhost:3000/health

Revisa la conexión a base datos

Request

/health

Host: localhost:3000

Response Exitosa

```
{  
  "status": "healthy",  
  "database": "connected",  
  "timestamp": "2025-12-08T00:22:01.604Z"  
}
```

La API no requiere autenticación (usa usuario ID=1 por defecto). En producción se debe implementar:

- JWT (JSON Web Tokens)
- Middleware de autenticación
- Validación de permisos por usuario